LAB 8 (SECJ1023)

PROGRAMMING TECHNIQUE II

SECTION 03, 04 & 06, SEM 2, 2025/2026

1. Find all errors in each of the following fragments of code.

a.
```
class A {
    public:
        virtual myFunA() = 0;
        { cout << "Salam"; }
};
```

b.
```
class Base1 {
    public:
        int a;
        Base1(int b)
        { a = b; }
};

class Base2 {
    public:
    int c;
    Base2(int b)
    { return c = b; }
};

class Derived1: public Base1, public Base2 {
    public:
    int x;
};
```

c.
```
class BaseA {
    public:
    int a;
    BaseA(int b)
    { a = b; }
};

class BaseB: virtual public BaseA {
    public:
    int c;
    BaseB(int d): BaseA(d*d) {c = d;}
};

class BaseC: public BaseB {
    public:
    int e;
    BaseC(int f): BaseA(f*f*f) {e = f;}
};
```

2. What will the following programs display? In your own words, explain why such output is printed.

a.

```
 1  //Program 8.1
 2  #include <iostream>
 3  using namespace std;
 4
 5  class Class1
 6  {
 7      protected:
 8         int a;
 9      public:
10         Class1(int x = 1)
11         { a = x; }
12         int getVal()
13         { return a; }
14  };
15
16  class Class2 : public Class1
17  {
18     private:
19        int b;
20     public:
21        Class2(int y = 5)
22        { b = y; }
23        int getVal()
24        { return b; }
25  };
26
27  int main()
28  {
29     Class1 object1;
30     Class2 object2;
31     cout << object1.getVal() << endl;
32     cout << object2.getVal() << endl;
33     return 0;
34  }
```

b.

```
 1  //Program 8.2
 2  #include <iostream>
 3  using namespace std;
 4
 5  class Class1
 6  {
 7      protected:
 8         int a;
 9      public:
10         Class1(int x = 1)
11         { a = x; }
12          void change()
13         { a *= 2; }
14         int getVal()
15         { change(); return a; }
16  };
```

```
17
18  class Class2 : public Class1
19  {
20      private:
21         int b;
22      public:
23         Class2(int y = 5)
24         { b = y; }
25         void change()
26         { b *= 10; }
27  };
28
29  int main()
30  {
31      Class1 object1;
32      Class2 object2;
33      cout << object1.getVal() << endl;
34      cout << object2.getVal() << endl;
35      return 0;
36  }
```

c.

```
1   //Program 8.3
2   #include <iostream>
3   using namespace std;
4
5   class Class1
6   {
7       protected:
8          int a;
9       public:
10         Class1(int x = 1)
11         { a = x; }
12         virtual void change()
13         { a *= 2; }
14         int getVal()
15         { change(); return a; }
16  };
17
18  class Class2 : public Class1
19  {
20      private:
21         int b;
22      public:
23         Class2(int y = 5)
24         { b = y; }
25         virtual void change()
26         { b *= 10; }
27  };
28
29  int main()
30  {
```

```
    Class1 object1;
    Class2 object2;
    cout << object1.getVal() << endl;
    cout << object2.getVal() << endl;
    return 0;
}
```

d.

```
//Program 8.4
#include <iostream>
using namespace std;

class Base
{
    protected:
       int baseVar;
    public:
       Base(int val = 2)
       { baseVar = val; }
       int getVar()
       { return baseVar; }
};

class Derived : public Base
{
    private:
       int derivedVar;
    public:
       Derived(int val = 100)
       { derivedVar = val; }
       int getVar()
       { return derivedVar; }
};

int main()
{
    Base *optr;
    Derived object;
    optr = &object;
    cout << optr->getVar() << endl;
    return 0;
}
```

3. Write the declaration for class `B`. The class's members should be

   a. **`m`**, an integer. This variable should not be accessible to code outside the class or to member functions in any class derived from class `B`.

   b. **`n`**, an integer. This variable should not be accessible to code outside the class, but should be accessible to member functions in any class derived from class **`B`**.

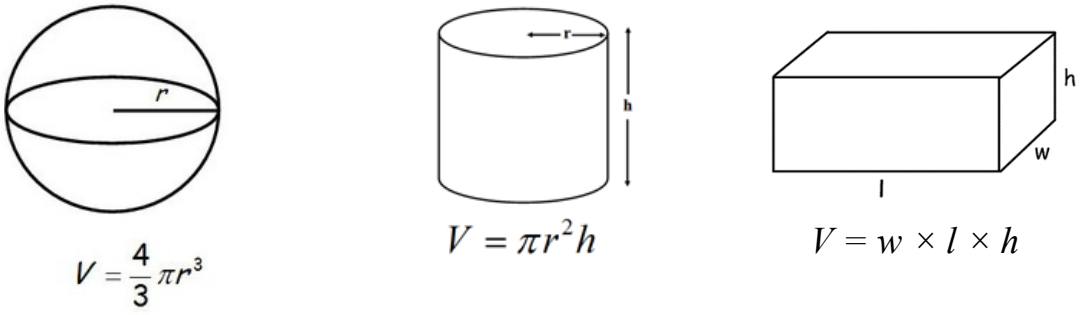
   c. **`setM`**, **`getM`**, **`setN`**, and **`getN`**. These are the set and get functions for the member variables **`m`** and **`n`**. These functions should be accessible to code outside the class.

d. **`calc`**, a public virtual member function that returns the value of **`m`** times **`n`**.

Next write the declaration for class **`D`**, which is derived from class **`B`**. The class's members should be

a. **`q`**, a float. This variable should not be accessible to code outside the class but should be accessible to member functions in any class derived from class **`D`**.
b. **`r`**, a float. This variable should not be accessible to code outside the class, but should be accessible to member functions in any class derived from class **`D`**.
c. **`setQ`**, **`getQ`**, **`setR`**, and **`getR`**. These are the set and get functions for the member variables **`q`** and **`r`**. These functions should be accessible to code outside the class.
d. **`calc`**, a public member function that overrides the base class **`calc`** function. This function should return the value of **`q`** times **`r`**.

4. Three dimensional (3D) objects can be represented with their geometrical characteristics. For example, a sphere can be defined by its radius, a cylinder with radius and height, and a cuboid with its dimension's width, length and height. As different attributes are used for representing the objects, the calculation to obtain the volume of each object is also different. Figure 8.1 shows the formulas to determine the volume for each object respectively. While Figure 8.2 shows the class diagram representing the relationship between these 3D objects as well as their attributes and methods.



(a) Sphere (b) Cylinder (c) Cuboid

Figure 8.1: Formula to obtain the volume of a sphere, cylinder and cuboid

Given an incomplete implementation for the class **`Sphere`**, **`Cylinder`** and **`Cuboid`**, in Program 8.5. Complete the program based on the tasks stated in the program (Task 1 to Task 8). Figure 8.3 shows an example output that your program should produce. Yours might be different if you use different values for each object.

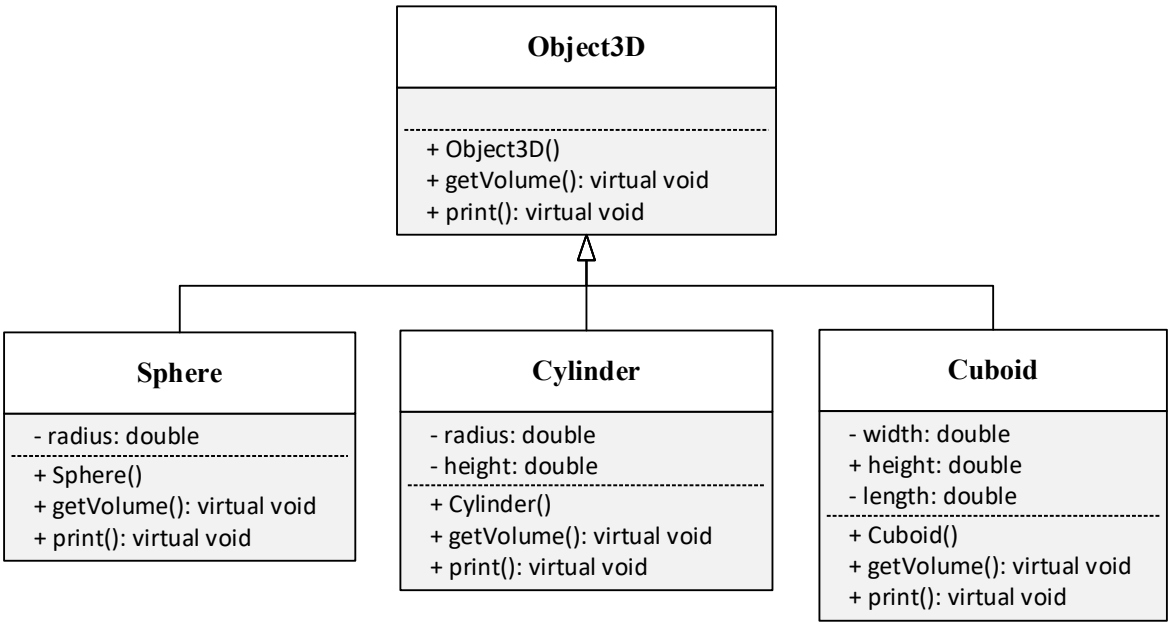


Figure 8.2: UML class diagram for three dimensional objects

```
Object #1
Cuboid: dimension= 10 x 20 x 30
Volume= 6000

Object #2
Cylinder: r=20, h=20
Volume= 25132

Object #3
Sphere: r=10
Volume= 4188.67

Object #4
Cylinder: r=2, h=5
Volume= 62.83

Object #5
Sphere: r=3
Volume= 113.094

Total volume = 35496.6
```

Figure 8.3: Expected output

```
 1  //Program 8.5
 2  #include<iostream>
 3  using namespace std;
 4
 5  #define PI 3.1415
 6  // Task 1: Complete the definition of the parent class
 7  //'Object3D'. Allow two methods below to be polymorphic
 8  //(i.e, make those methods to be virtual methods)
 9  //    + void print() const;
10  //    + double getVolume() const;
```

```
11
12  class Object3D {
13     public:
14        Object3D(){}
15        void print() const {}
16        double getVolume()const {return 0;}
17  };
18
19  // Task 2: Complete the definition of the class 'Sphere'.
20  // Redefine the following methods accordingly.
21  //    + void print() const; //to print the radius of the
22  sphere
23  //    + double getVolume() const; //to calculate the volume of
24                                     // the sphere
25
26  class Sphere : public Object3D {
27     protected:
28        double r;
29     public:
30        Sphere(double _r=0){setRadius(_r);}
31        void setRadius(double _r){ r = _r;}
32        double getRadius()const {return r;}
33        void print() const {cout << "Sphere: r=" << endl;}
34        double getVolume() const {}
35  };
36
37  // Task 3: Complete the definition of the class 'Cylinder'.
38  // Redefine the following methods accordingly.
39  //    + void print() const; //to print the radius and
40                              //height of the cylinder
41
42  //    + double getVolume() const; //to calculate the volume of
43                                     //the cylinder
44
45  class Cylinder : public Object3D {
46     protected:
47        double r,h;
48     public:
49        Cylinder(double _r=0, double _h=0){
50           setRadius(_r);
51           setHeight(_h);
52        }
53
54        void setRadius(double _r){ r = _r; }
55        void setHeight(double _h){ h = _h; }
56        double getRadius()const {return r;}
57        double getHeight()const {return h;}
58
59        void print() const{
60           cout << "Cylinder: r=" << ", h=" << endl;
61        }
62
63        double getVolume() const {}
```

```
};


// Task 4: Complete the definition of the class 'Cuboid'.
// Redefine the following methods accordingly.
//   + void print() const; //to print the width, length,
                           //and height of the cuboid
//   + double getVolume() const; //to calculate the volume of
                                 //the cuboid

class Cuboid : public Object3D {
    protected:
       double w,l,h;
    public:
       Cuboid(double _w=0, double _l=0, double _h=0){
           setWidth(_w);
           setLength(_l);
           setHeight(_h);
       }

       void setWidth(double _w){ w = _w;}
       void setLength(double _l){ l = _l;}
       void setHeight(double _h){ h = _h;}
       double getWidth()const {return w;}
       double getLength()const {return l;}
       double getHeight()const {return h;}

       void print() const{
          cout << "Cuboid: dimension = "
               << w << " x " << l << " x " << h << endl;
       }

       double getVolume() const {}
};

int main( )
{
  // Task 5: Create an array of pointers of Object3D.

  // Task 6: Fill in the array with different types of
  // objects, for example cuboids, spheres and cylinders.
  // Put all the objects in the same array.

  // Task 7: Print the information of all objects along with
  // their volume

  for (int i=0; i<5; i++){
      cout << "Object #" << (i+1) << endl;
      // print object here
      cout << "Volume= " <<   endl;
      cout << endl;
  }

```

```
117     // Task 8: Calculate and print the total volume of the
118     // objects
119
120     double totalVolume = 0;
121
122     // do summation of volume using a loop here
123
124     cout << "Total volume = " << totalVolume << endl;
125     return 0;
126 }
```

5. Write a complete C++ program for **`Transport`**, **`Car`** and **`Motor`** classes based on the following specification:

   a. Design a **`Transport`** class that has the following members:

      i. A member variable for the name of the transport (a **`string`**)

      ii. A member variable for the year that the transport was built (a **`string`**)

      iii. A constructor and appropriate accessors and mutators.

      iv. A virtual **`print`** function that displays the transport's name and the year it was built.

   b. Design a **`Car`** class that is derived from the **`Transport`** class. The **`Car`** class should have the following members:

      i. A member variable for the maximum number of passengers (an **`int`**)

      ii. A constructor and appropriate accessors and mutators

      iii. A **`print`** function that overrides the print function in the base class. The **`Car`** class's print function should display only the car's name and the maximum number of passengers.

   c. Design a **`Motor`** class that is derived from the **`Transport`** class. The Motor class should have the following members:

      i. A member variable for the maximum number of passengers (an **`int`**)

      ii. A constructor and appropriate accessors and mutators.

      iii. A **`print`** function that overrides the print function in the base class. The **`Motor`** class's print function should display only the motor's name and the maximum number of passengers.

   d. Write a **`main()`** method that has an array of Transport pointers. The array elements should be initialized with the addresses of dynamically allocated **`Transport`**, **`Car`**, and **`Motor`** objects. The program should then step through the array, calling each object's **`print`** function.

6. Define a pure abstract base class called **`BasicShape`**. The **`BasicShape`** class should have the following members:

   a. Private member variable:

      `area`, a double used to hold the shape **`s`** area.

   b. Public member functions:

i. **`getArea`**. This function should return the value in the member variable area.

ii. **`calcArea`**. This function should be a pure virtual function.

c. Next, define a class named **`Circle`**. It should be derived from the **`BasicShape`** class.

d. It should have the following members:

i. Private member variables:

- **`centerX`**, a long integer used to hold the **`x`** coordinate of the circle's center.
- **`centerY`**, a long integer used to hold the **`y`** coordinate of the circle's center.
- **`radius`**, a double used to hold the circle's radius.

ii. Public Member Functions:

- constructor accepts values for **`centerX`**, **`centerY`**, and **`radius`**. Should call the overridden **`calcArea`** function described below.
- **`getCenterX`** returns the value in **`centerX`**.
- **`getCenterY`** returns the value in **`centerY`**.
- **`calcArea`** calculates the area of the circle (area = 3.14159 * radius * radius) and stores the result in the inherited member area.

e. Next, define a class named **`Rectangle`**. It should be derived from the **`BasicShape`** class. It should have the following members:

i. Private member variables:

- **`width`**, a long integer used to hold the width of the rectangle.
- **`length`**, a long integer used to hold the length of the rectangle.

ii. Public member functions:

- constructor accepts values for width and length. Should call the overridden
- **`calcArea`** function described below.
- **`getWidth`** returns the value in **`width`**.
- **`getLength`** returns the value in **`length`**.
- **`calcArea`** calculates the area of the rectangle (area = length * width) and stores the result in the inherited member area.

After you have created these classes, create a driver program that defines a **`Circle`** object and a **`Rectangle`** object. Demonstrate that each object properly calculates and reports its area.